



GitProjectHealth: an Extensible Framework for Git Social Platform Mining

Nicolas Hlad, Benoit Verhaeghe, Kilian Bauvent

► To cite this version:

Nicolas Hlad, Benoit Verhaeghe, Kilian Bauvent. GitProjectHealth: an Extensible Framework for Git Social Platform Mining. 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Apr 2025, Ottawa, France. pp.596-600, <10.1109/MSR66628.2025.00094>. <hal-05127341>

HAL Id: hal-05127341

<https://hal.science/hal-05127341v1>

Submitted on 24 Jun 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

GitProjectHealth: an Extensible Framework for Git Social Platform Mining

Nicolas Hlad
Berger-Levrault

Toulouse, France

nicolas.hlad@berger-levrault.com

0000-0003-4989-2508

Benoit Verhaeghe
Berger-Levrault

Lyon, France

benoit.verhaeghe@berger-levrault.com

0000-0002-4588-2698

Kilian Bauvent
Berger-Levrault

Lyon, France

kilian.bauvent@berger-levrault.com

Abstract—Git social platforms (such as Gitlab, Github, or Bitbucket) provide insight into a team’s workflow. Mining Software Repositories (MSR) provides methods and tools to extract data from these platforms. However, most tools lack connectivity and extensibility across multiple platforms. Moreover, they rarely connect to other project management platforms such as Jira. In this paper, we introduce GitProjectHealth (GPH), a framework to extract data from any Git repositories and social platforms. GPH is implemented inside a model-driven engineering framework in Pharo smalltalk, facilitating its extension to other social platforms. We demonstrate GPH features over 3 open-source organizations: Eclipse, MooseTechnology and Microsoft; as well as Berger-Levrault, a closed-source company. We extracted their activity to build distributions of commits by user and to determine which types of ticket were associated with each merge request.

Index Terms—Mining Software Repositories, Model Driven Engineering, Pharo, Moose

I. INTRODUCTION

Mining Software Repositories (MSR) refers to processes for analyzing software repository data to gain insights into software development processes. MSR focuses on extracting information from version control systems like Git to understand aspects such as bug introductions, code complexity, and developers’ contribution activity over time [1].

Git social platforms such as Gitlab, Gitlab, or Bitbucket have introduced interaction between developers [2] with Merge Request, Code Review, and CI/CD capabilities. Their features provide new information about the developer’s workflow and the project itself. New tools have been proposed to analyze this information [3], and to create automated processes to produce code [4].

External tools, along with Git and existing Git social platforms, also play a crucial role in shaping the development team environment and must be considered to gain a comprehensive overview of development activity. For example, the Atlassian Jira product is widely used by development teams to manage tickets and plan the development, but is not considered when analyzing a git repository or git social platforms.

As a result, MSR engineers are faced with a diversity of data veins (Git and project management platforms), each requiring its own set of tools to be mined. We observe a lack of common and generic tools for better understanding the behavior of developers.

In Software Engineering, Model-Driven Engineering (MDE) uses models as a primary artifact to abstract the development process [5]. For instance, MDE approaches abstract multiple codebases of different languages to refactor, maintain and evolve them [6, 7]. Furthermore, software analysis platforms such as Moose [8] offer generic tools to perform navigation and refactoring inside these models. Thus, an MSR approach could benefit from MDE by 1) abstracting a single model for any Git social platforms; and 2) to analyze them reusing generic algorithms found in platforms like Moose, e.g. for navigating among the model entities.

In this paper, we present GitProjectHealth (GPH), an MDE-based tool for multi-Git platform mining and analysis. GPH [9] contains an extensible metamodel for Git, Git social platforms, and that extends to ticket management platforms like Jira. GPH also comes as a tool that (1) extracts data from major social platforms, (2) imports a model of a specific Git entity (a project, a user, a commit, etc.), (3) makes connection between models (i.e. Gitlab and Jira), and (4) allows to visualize the model’s data, navigate it and compute metrics. GitHealthProject is open source and customizable at runtime, so anyone can contribute with their own visualizations, import new data or add their own metrics.

We experimented with GPH on open-source and closed-source projects for Berger-Levrault, an international company selling software systems. For closed-source projects, we demonstrate the connection with Atlassian’s Jira system by collecting tickets associated with merge requests. We were able to easily summarize which types of ticket each merge request is associated with, thus gaining insight on the development activity.

The paper is structured as followed: Section II presents related work on MSR and code analysis. Section III presents our tool and its approach to gather data and connect them. Section IV demonstrates how GPH mines closed-source and open-source projects, before concluding with section V.

II. RELATED WORK

PyDriller is a python tool proposed by Spadini et al. [1], that allows one to easily analyze the commits history of a git project. PyDriller wraps API requests for Git social platforms, e.g. to retrieve information about commits such

as author, date, line modified, and more. It also integrates some common algorithms such as SZZ to retrieve which commit has introduced a bug. Whereas this tool provides much information, it ignores the git social platform data that are required to perform a deep understanding of development behavior. Also, it cannot be easily integrated with other domain tool analyzers such as those that deal with generic code analysis.

Javapers is a tool proposed by Rukmono and Chaudron [10] that extracts a knowledge graph from Java source files, leveraging the Spoon library [11]. Javapers facilitates the manipulation and analysis of Java projects and can be integrated into CI pipelines to give insight to the development team. However, Javapers only considers the source code and leave beside information such as bug issues, or merge request information.

Pandora Nguyen et al. [12] is a tool to extract data from several platforms and to keep them for a long period. Pandora focus on data concerning continuous integration and deployment from platform such as Gitlab, SonarQube and Jenkins. It also stores access to data that are usually ephemeral in those platforms. However, there is no integrated solution for matching data from different sources, i.e. ticket system, which reduced the opportunity of any combined analysis.

Recently, Varanasi and Karre [13] proposed to focus the analysis on detecting the wasted time by developers. To do so, they use the Gitlab REST API to extract information relative to existing forks, stale branches and rejected Merge Requests. Their work enables practitioners to focus directly on the development problems instead of looking for metrics relative to the velocity of the development. This work only investigates data from the Gitlab git social platform and could be extended to other git platforms as well as project management systems to benefit most industrial setups.

Industrial software systems such as LinearB¹ or Swarmia² propose a plug-and-play solution to extract information from Git Social Platforms and ticket management system like Jira. They also enable features to notify a developer when they detect deviant behavior like larger than usual merge request. These tools offers teams to import their own metrics. However, they rarely let the teams build new metrics by collecting their data. Finally, they exploit the connection between platforms but data are closed to further analysis (black-box).

III. GITPROJECTHEALTH FRAMEWORK

GitProjectHealth is developed in Pharo [14] and can be easily installed using the last stable version of Moose (version 11) downloadable using the Pharo Launcher³. Moose is an existing ecosystem of tools builds in smalltalk Pharo, to create, navigate, manipulate and connect entities of any moose model [8]. After installation, it only requires a private token for Git social platforms and the organization ID for analysis. Next, we present an overview on the extraction process of GPH with an

excerpt of the extracted data and we connect models together with GPH.

A. Extraction process

GPH process is illustrated in fig. 1 and has 4 phases:

1) Extracting data : From a Pharo script made by a user, GPH can extract data from the different platforms (using the platform URL and a token). Optionally, GPH works with a *time period* and a *time scale*. A time period corresponds to two dates (*since* and *until*) allowing GPH to run over an interval of time. A time scale specifies the granularity of the analysis, to average results over days, weeks, months, or years. This flexibility enables us to fine-tune the analysis views, tailoring it to short-term trends or long-term patterns as needed.

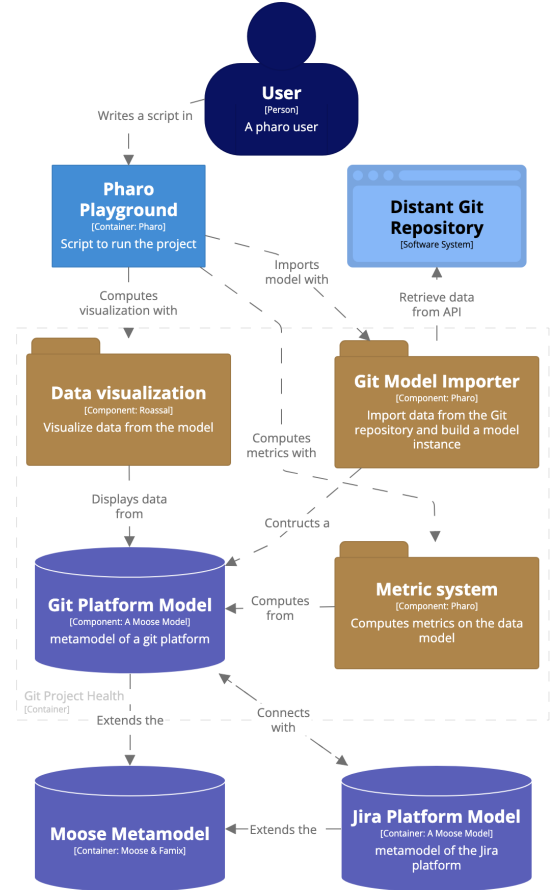


Fig. 1: C4 model of GitProjectHealth

2) Importing models: Regardless of the repository analyzed by GPH (e.g., Gitlab, Github, or Bitbucket), GPH generates a model that is always an instance of GPMModel⁴. We present GPMModel in Section III-B. For Jira, we dedicate a specific model called JPMModel⁵.

3) Connecting metamodels: Then, GPH connects the Git model with the Jira model (see section III-C). We include model connector in our projects, which e.g. connect a Jira

¹<https://linearb.io/>

²<https://www.swarmia.com/>

³Getting started with GPH: https://youtu.be/BIOF_mj6dDk

⁴shortened from *Git Social Platform*

⁵shortened from *Jira Platform Model*

issue with commits, and *diff* information of commits with a complete project modelization.

4) Metrics and visualizations: Finally, we integrate pre-built visualizations to obtain project charts, as well as a module to compute metrics. We present some visualizations in section IV.

B. Git Social Platform Metamodel

We built the GPH metamodels within the Moose platform. Moose also comes with pre-packaged metamodels which enables us to quickly analyze most existing Java, Python, Fortran, and TypeScript projects. In our context, we built two main metamodels: one to represent git social platform data hosted by the GPH project and another to model Jira ⁶.

Figure 2 presents a partial representation of the entities we implemented. The concepts are based on Gitlab and Gitlab definition, but extend to Bitbucket⁷. The complete metamodel is available on the project website⁸. First, the Group concept represents a group of projects and it is a recursive definition that allows sub-group. A Project corresponds to a git project, the page used to access the Repository, Merge Requests, and other concepts. The Repository is the git environment with its associations to Commits and Branches. In our model, each Commit is associated with a User who is the author or the committer. A commit is also linked to a Diff entity that can be explored through DiffRange and Changes to compute metrics such as code churn or code comment. Finally, a User is also linked to the Job it started inside a Pipeline for project analysis of the code.

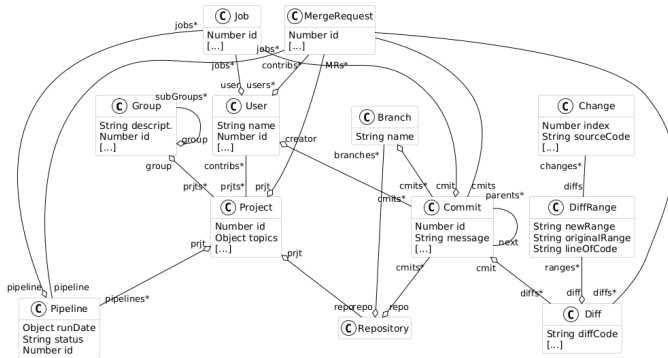


Fig. 2: Simplified UML of our Git Platform metamodel

C. Connecting Models

The metamodel connection is a complex topic⁹ that we only summarize in this section. We connect metamodels with Moose's ability to create relations between concepts (i.e. entity definition) from two different metamodels [15]. To do

so, we define a *metamodel-connector*, that uses other metamodels as *submetamodels*. A connector can access the remote concepts declared in its submetamodels. For instance, the connector between our Git model and a Jira model accesses the remote definitions of a Merge Request (MR) concept in GPHModel, and of a Atlassian Ticket concept in JPMModel. To link the MR to the ticket, the connector declares a new relation between these two remote concepts. Figure 3 demonstrates the extraction of a work duration by mapping Jira tickets to Git commits in the model representations. In this example, 12 days were spent on ticket JIRA-56 that are linked to commit 1234. This highlights one custom way to retrieve external information through the metamodel connection mechanism.

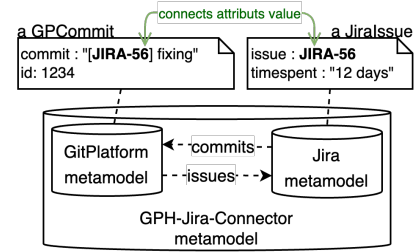


Fig. 3: Illustrating metamodels connection

IV. EXPERIMENTATION

In this section, we apply our tool on open-source project groups of 3 organizations: *Eclipse*, *Microsoft* and *MooseTechnology*. We also apply it on a private company repository to showcase our model connector and gather data from their git social platform and their Jira platform.

A. An open-source reproducible analysis

To perform this analysis, we perform the script in listing 1. We analyze 30 days of activities from 23 August 2024 to 23 September 2024. It took around 06:36 minutes to collect data from Gitlab and to build a model of over 4457 entities. Among them, we mine 340 projects distributed as: 1 microsoft; 205 eclipse and 81 moosetechnology. We retrieve 1899 commits distributed over 1829 open branches and managed by 152 users.

Listing 1: Extract data from Eclipse Gitlab organisation

```
1 glhModel := GPHModel new.
2 githubImport := GithubModelImporter new
3   glhModel: glhModel;
4   privateToken: '<YOUR_GITHUB_PRIVATE_TOKEN>';
5   withCommitsSince: (Date today - 30 days);
6   yourself.
7 githubImport importGroup: 'eclipse'.
8 githubImport importGroup: 'moosetechnology'.
9 githubImport importProject: 'vscode'
10   ofGroup: 'microsoft'.
11 glhModel
```

To visualize this analysis, we use the prebuilt visualization of GPH, that allows us to explore the commits' distribution. For instance, Figure 4 presents the commit's distribution over

⁶<https://github.com/Evref-BL/Jira-Pharo-API>

⁷E.g. Bitbucket definition of a project is equivalent to a Gitlab Group

⁸<https://modularmoosetech.org/moose-wiki/Users/gitproject-health/getting-started-with-gitproject-health.html>

⁹<https://modularmoosetech.org/posts/2021-05-15-connecting-metamodels#connect-remote-entities>

each day, from the 9th of September to the 13th of September of the vscode project. Each bar corresponds to the number of commits made at a date, and each color to a different committer.

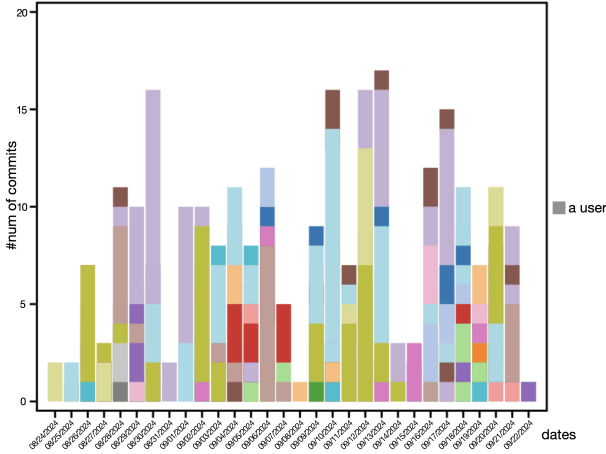


Fig. 4: Commit distribution for *vscode* during September 2024. Each color represents a user

We can also select a user in the GPH interface and visualize their activity on an organization. Figure 5 shows the commits' distribution of one user of the *moosetechnology* group during the month period. It highlights the working days and the most important projects for this person during this period, *i.e.*, *MooseIDE* in orange and *Moose* in blue.

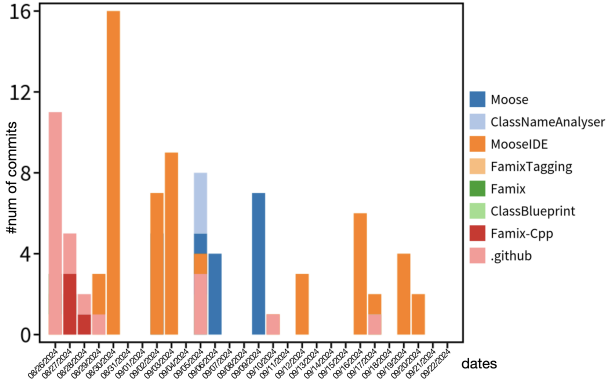


Fig. 5: A user's commit activity by day, during the month of September 2024, on *moosetechnology*

B. An industrial use-case

We ensure that GPH works in complex environments, such as those used in industrial companies. Here we are interested in connecting a Jira model with a Git model. We tested it at Berger-Levrault¹⁰. They are an IT company that provides software solutions to the public sector and private stakeholders. Using GPH, we mined a group of 5 projects related to human resource management called WeHR and gives the

partial script¹¹ with comments in listing 2. We search the distributions of merge request (MR) associated with different types of Jira issue. This script echos the first steps explained earlier in Figure 3, to retrieve bug location from Jira to the source code. We link the data of the company's Gitlab instance to the Atlassian Jira instance (see section III-C). Note that there are several kinds of tickets: *Anomaly*, *Task*, *Epic*, and so on. Table I displays the script results. Here, 27% of the merge requests created during those 30 days are related to bug fixing issues (*i.e.*, *anomaly*).

Listing 2: Retrieve commits related to fixing bugs

```
1 "load a group from its Id"
2 grp := glhImporter importGroup: 1234.
3 "get merge requests (mrs) for all group's projects"
4 mrs := grp allProjectstoScope flatCollect: [:prjt |
5   "MR over the last thirty days"
6   glhImporter importMergeRequests: prjt
7   since: (Date today - 30 day)
8   until: Date today ]. "seventy MRs found"
9 "select MR with an issues"
10 mrIssues := mrs select: [:mr | mr jiraIssue isNotNil].
11 (mrIssues collect: [:mr | mr jiraIssue type name]) asBag.
```

In summary, we highlight GPH's ability to handle real-world data in both open-source and enterprise contexts. Through the open-source analysis, we successfully collected and modeled data from 3 organizations, visualizing their user activity and project commits over time. Furthermore, we validate our approach through an industrial use-case, by integrating Jira and Git data to analyze merge request distributions across different issue types.

Type	Occurrences	Occurrences %
Anomaly	19	27%
Task	21	30%
Epic	26	37%
None	4	6%
Total	70	100%

TABLE I: Issues occurrences by type for WeHR

V. CONCLUSION

In this paper, we introduce GitProjectHealth, an extensible framework for mining data from Git social platforms build on top of Model Driven Engineering. Our framework addresses the need for comprehensive insight into development workflows by connecting data from git repositories, social platforms, and ticket management systems. Through our hands-on on both open-source and industrial projects, we experiment GPH's ability to provide detailed analyses, such as identifying bug-prone modules and developers' activities. In future work, we will focus on enhancing the framework's capabilities and expanding its compatibility with additional tools and platforms. We will also investigate building connected visualization tools allowing one to navigate comprehensively across data from multiple platforms.

¹¹full script: <https://modularmoose.org/moose-wiki/Users/gitproject-health/getting-started-with-gitproject-health.html#jira-connector>

¹⁰<https://www.berger-levrault.com>

REFERENCES

- [1] D. Spadini, M. Aniche, and A. Bacchelli, “PyDriller: Python framework for mining software repositories,” in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3236024.3264598>
- [2] L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, “Social coding in GitHub: Transparency and collaboration in an open software repository,” in *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work*. ACM, 2012, pp. 1277–1286. [Online]. Available: <https://dl.acm.org/doi/10.1145/2145204.2145396>
- [3] G. Gousios, “The GHTorrent dataset and tool suite,” in *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 2013, pp. 233–236. [Online]. Available: <http://ieeexplore.ieee.org/document/6624034/>
- [4] N. Chidambaram, T. Mens, and A. Decan, “RABBIT: A tool for identifying bot accounts based on their recent GitHub event history,” in *Proceedings of the 21st International Conference on Mining Software Repositories*. ACM, 2024, pp. 687–691. [Online]. Available: <https://dl.acm.org/doi/10.1145/3643991.3644877>
- [5] D. C. Schmidt *et al.*, “Model-driven engineering,” *Computer-IEEE Computer Society*-, vol. 39, no. 2, p. 25, 2006.
- [6] N. Hlad, B. Verhaeghe, and M. Derras, “Analyzing dart language with pharo: Report and early results,” in *International Workshop on Smalltalk Technologies 2023*, 2023.
- [7] M. Martinez and M. Monperrus, “Astor: A program repair library for java,” in *Proceedings of the 25th international symposium on software testing and analysis*, 2016, pp. 441–444.
- [8] N. Anquetil, A. Etien, M. H. Houekpetodji, B. Verhaeghe, S. Ducasse, C. Toullec, F. Djareddir, J. Sudich, and M. Derras, “Modular moose: a new generation of software reverse engineering platform,” in *Reuse in Emerging Software Engineering Practices: 19th International Conference on Software and Systems Reuse, ICSR 2020, Hammamet, Tunisia, December 2–4, 2020, Proceedings 19*. Springer, 2020, pp. 119–134.
- [9] B. Verhaeghe, N. Hlad, K. Bauvent, and N. Anquetil, “GitProjectHealth,” Oct. 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.13886505>
- [10] S. A. Rukmono and M. R. Chaudron, “Enabling analysis and reasoning on software systems through knowledge graph representation,” in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 120–124.
- [11] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, “Spoon: A library for implementing analyses and transformations of java source code,” *Software: Practice and Experience*, vol. 46, no. 9, pp. 1155–1179, 2016.
- [12] H. Nguyen, F. Lomio, F. Pecorelli, and V. Lenarduzzi, “PANDORA: Continuous Mining Software Repository and Dataset Generation,” in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 263–267. [Online]. Available: <https://ieeexplore.ieee.org/document/9825866/>
- [13] D. S. Varanasi and S. A. Karre, “Measuring Software Development Waste in Open-Source Software Projects,” *IEEE SEAA*, p. 9, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2409.19107>
- [14] M. Denker, S. Ducasse, S. V. Caekenberghe, N. Hartl, G. Polito, and E. Lorenzano, “Pharo - welcome to pharo!” [Online]. Available: <https://pharo.org/>
- [15] G. Darbord, F. Vandewaeter, A. Etien, N. Anquetil, and B. Verhaeghe, “Modest-Pharo: Unit Test Generation for Pharo Based on Traces and Metamodels,” in *IWST 2024: International Workshop on Smalltalk Technologies*, Lille, France, Jul. 2024. [Online]. Available: <https://hal.science/hal-04622256>